STAT3612 · FALL 2026 · TUTORIAL

Machine Learning for Time Series Data

From Data Structures to Learned Representations

Tutor: Yinghao Zhu
yhzhu99@connect.hku.hk

Tutorial outline

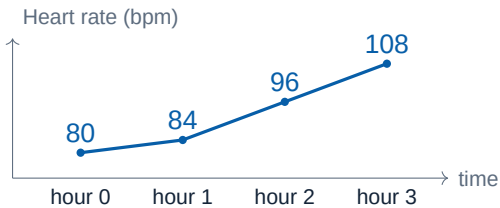
1. Data	Time series data: adding a time dimension
2. Tasks	What can we predict or detect from a time series?
3. Features	How can we turn a history into useful features?
4. Models	Can a model learn temporal patterns for us?
5. Evaluation	How do we know our predictions are useful?
6. Group project	Predict a two-week learning-platform pause.

Time series data: adding a time dimension

Tabular data

Patient	Age	Heart rate
A	65	80
B	42	92
C	58	76

Time series: patient A



Observations indexed by time: values, order, and intervals.

Data structures: rows, time steps and channels

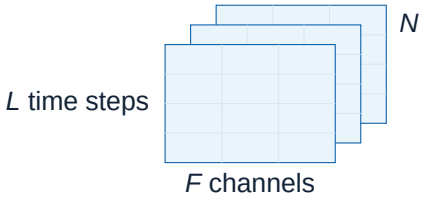
Tabular: $\mathbf{Z} \in \mathbb{R}^{N \times D}$

Patient	Mean HR	Latest HR	Change
A	90	120	$+60$
B	90	60	-60
C	80	80	0

N examples; D features per example.

Summaries or static attributes in each row.

Sequences: $\mathbf{X} \in \mathbb{R}^{N \times L \times F}$



One patient: $L \times F$.

100 patients, 24 hours, 3 vitals: $100 \times 24 \times 3$.

Univariate $F = 1$: $N \times L \times 1$, often stored as $N \times L$.

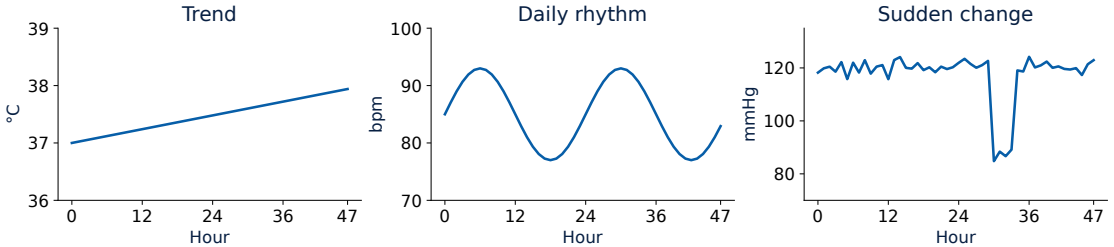
Time series tasks: prediction and detection

Shared input $\mathbf{X} \in \mathbb{R}^{N \times L \times F}$: observed histories.

Task	Example question	Prediction output
Forecasting	Next H hours of K vital signs?	$\widehat{\mathbf{Y}} \in \mathbb{R}^{N \times H \times K}$
Classification	Will a future event occur?	$\widehat{\mathbf{p}} \in [0,1]^{N \times 1}$ (binary)
Sequence labeling	What state is each observed hour in?	$\widehat{\mathbf{P}} \in [0,1]^{N \times L \times C}$
Anomaly detection	Which observed hours are unusual?	$\mathbf{S} \in \mathbb{R}^{N \times L}$ (scores)

Output axes H: forecast horizon; K: forecast channels; C: state classes.

Characteristics: order and temporal patterns

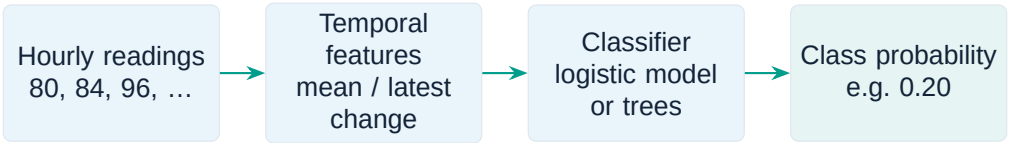


Temporal order Which reading came first? Shuffling can change the meaning.

Trend A sustained rise or fall; one possible pattern in that order.

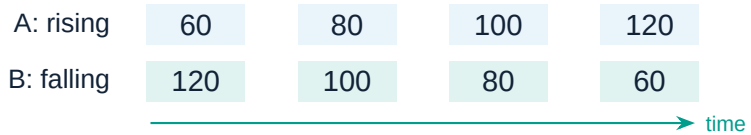
Other patterns Seasonality, shocks and dependence on recent values.

Modeling approach: from sequences to features



Which feature distinguishes rising from falling?

Feature engineering: level, recency and change

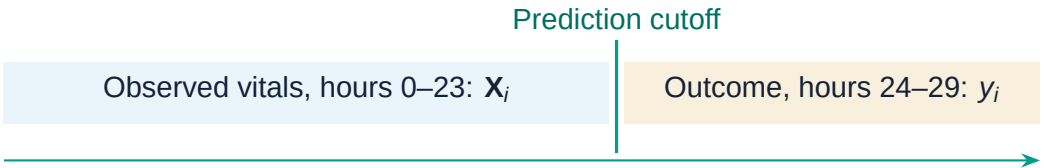


Feature	A	B	What does it tell us?
Mean	90	90	Overall level; loses order
Latest value	120	60	Where the series is now
Latest minus earliest	$+60$	-60	Direction and size of change

Feature engineering: temporal summaries

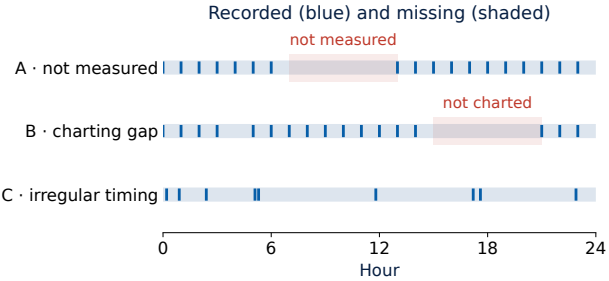
Overall level	Mean heart rate over the observed window.
Recency	Time since last measurement; latest value.
Change	Recent 6-hour mean minus previous 6-hour mean.
Variability	Standard deviation; hour-to-hour changes.
Extremes	Maximum heart rate; minimum blood pressure.
Context	Age; admission type.

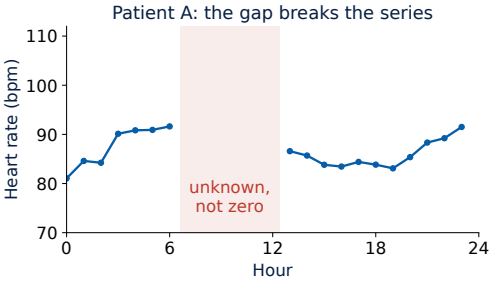
Task definition: input window and target



Input	Vital signs available by the end of hour 23.
Target	Sepsis onset during hours 24–29.
Cohort	Patients without sepsis at the cutoff.

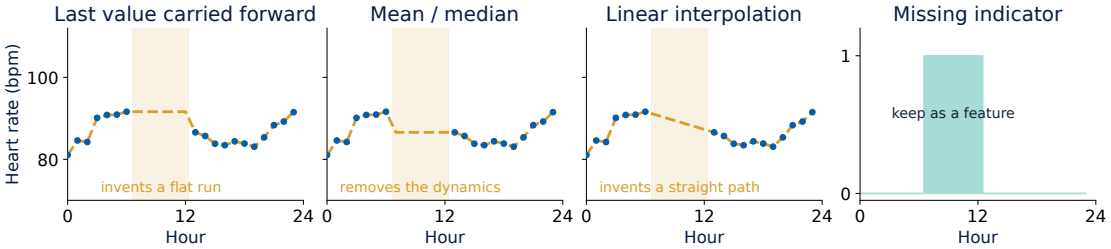
Preprocessing: where gaps come from





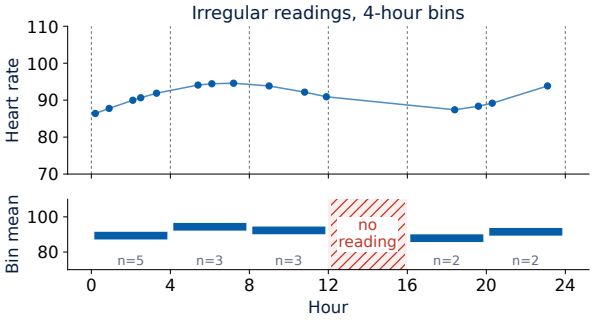
Not measured	No reading or test ordered; the gap itself can be informative.
Charting gap	The value exists but was not entered; check for a pattern.
Irregular timing	Readings and clicks arrive at event times, not on a fixed grid.

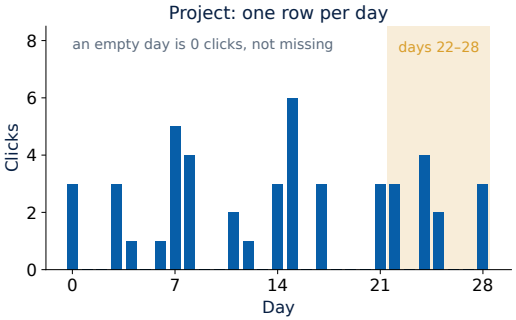
Preprocessing: filling one gap



LOCF	Keeps the last level; a flat run can hide a real change.
Mean / median	Simple and stable; erases the within-window pattern.
Interpolation	Smooth for short gaps; assumes a straight path across long ones.
Keep the trace	Add `was_missing` and time since the last measured value.

Preprocessing: irregular timing to a grid





Choose the grid	Hourly for vitals, daily for the project; half-open bins.
Aggregate	Per bin: count, sum, last, mean, max; time since the last event.
Empty bin	Zero clicks in an event log; unknown for a measurement.

Preprocessing in pandas

Events → daily grid

```
# one row per sample, day, type
daily = vle.pivot_table(
    index="sample_id",
    columns="day", values="clicks",
    aggfunc="sum", fill_value=0)
daily = daily.reindex(
    range(29), axis=1,
    fill_value=0)   # empty = 0
```

Readings → hourly grid

```
grid = pd.date_range(
    start, periods=24,
    freq="h")
hr = hr.reindex(grid)  # NaN
ffill = hr.ffill(limit=2)
interp = hr.interpolate(
    limit=3)
missing = hr.isna().astype(int)
```

Development data

Explore fills and scaling using train / validation; keep test independent.

Feature engineering in Python

```
# hr: one row per patient, one column per hour (0, ..., 23)
# Heart rate in bpm; NaN = not measured
features = pd.DataFrame(index=hr.index)
features["mean_hr"] = hr.mean(axis=1)
features["max_hr"] = hr.max(axis=1)
features["last_6h"] = hr.loc[:, 18:23].mean(axis=1)
features["previous_6h"] = hr.loc[:, 12:17].mean(axis=1)
features["change"] = (
    features["last_6h"] - features["previous_6h"]
)
```

Can we use hour 24?

Classifiers: logistic regression and trees

Logistic regression

$$p_i = \sigma(b + \mathbf{w}^\top \mathbf{z}_i), \qquad \sigma(a) = \frac{1}{1 + e^{-a}}.$$

Linear score + sigmoid.

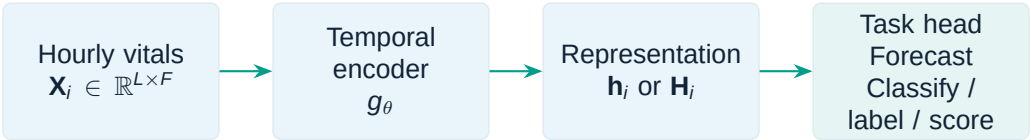
Tree-based classifiers

Nonlinear rules and interactions.

Random forest / gradient boosting.

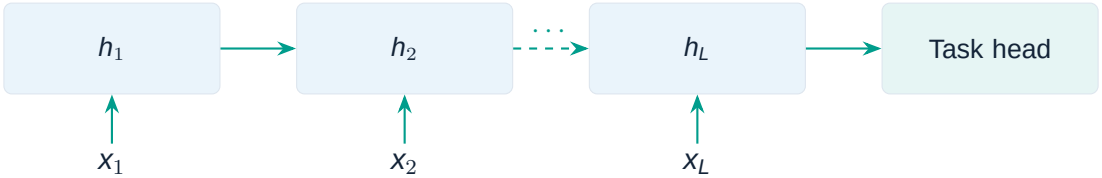
First baseline Predict the training event rate for everyone.

Sequence models: learning a representation



Representation	Pool to one vector $\mathbf{h}_i$, or keep time-step vectors $\mathbf{H}_i$.
Task head	Map the representation to the required output shape.
Training objective	Forecast error, classification loss, or an anomaly-learning objective.

RNN: recurrent memory

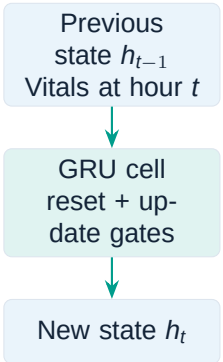


$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b).$$

Hidden state h_t: memory of the sequence so far.

Limitation Vanishing / exploding gradients on long sequences.

GRU: gated memory



Reset gate

Past information used in the candidate state.

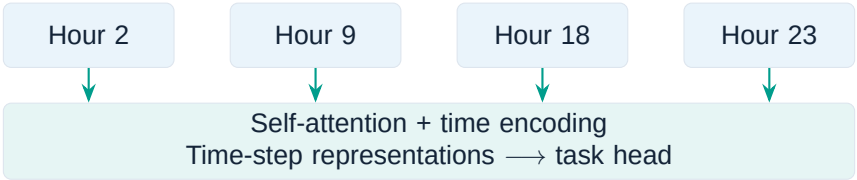
Update gate

Balance between old state and new candidate.

LSTM

Gated memory with a separate cell state.

Transformer: attention across time



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

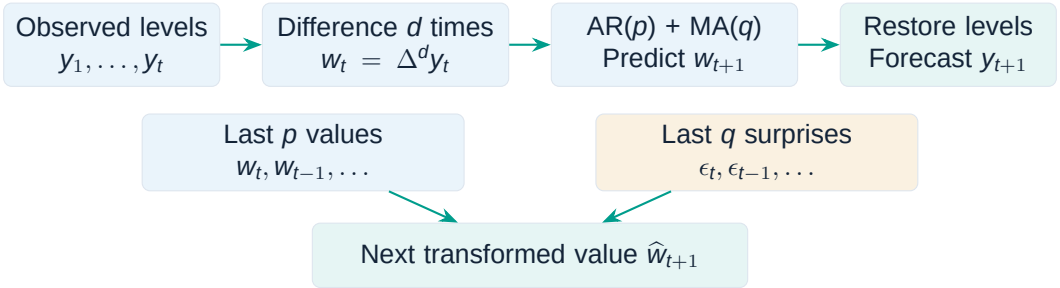
Intuition Query: what to find. Key: what to match. Value: what to combine.

Time order Add positional or time encodings.

Attention masks: observed vs. future inputs

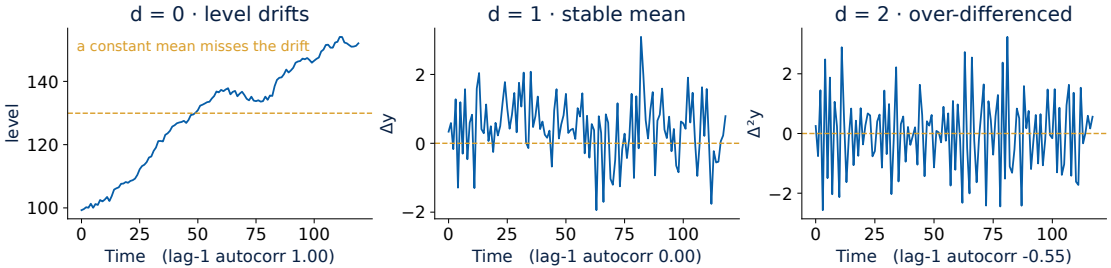
Whole window observed	Bidirectional attention within observed hours 0–23.
Alerting hour by hour	Causal masking: each hour sees only its past and present.
Forecasting vitals	Autoregressive training: shifted targets + causal masking.
Padding	Mask padded positions in variable-length sequences.

Forecasting: ARIMA (p, d, q) in pictures



Example: $(1, 1, 1)$ Predict the next change from the last change and last surprise.

Forecasting: what differencing does



$d = 0$ Model levels y_t directly.

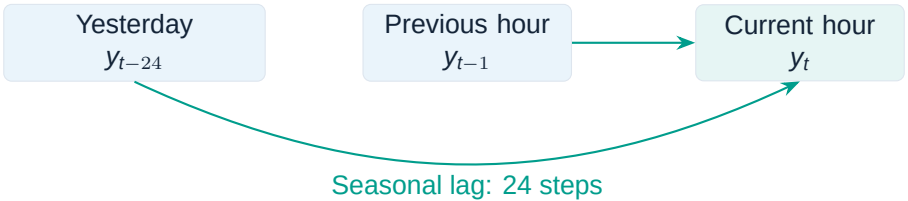
$d = 1$ Model changes $\Delta y_t = y_t - y_{t-1}$. Here the drift becomes stable.

$d = 2$ Model changes of changes. Here it adds unnecessary noise.

Forecasting: ARIMA and its variants

Model	What changes?	Example
AR / MA / ARMA	No differencing ($d = 0$)	Stable series
ARIMA (p, d, q)	Add differencing	A drifting level
SARIMA	Seasonal orders $(P, D, Q)_s$	Hourly data: daily cycle, $s = 24$
ARIMAX / SARIMAX	Add external predictors	Known calendar / schedules

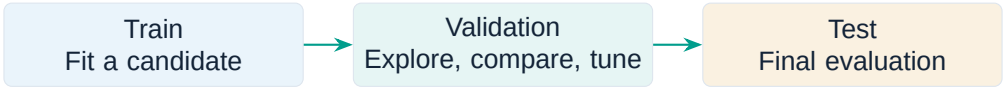
P, D, Q: seasonal versions of p, d, q; s: steps per cycle.



Model comparison: assumptions and trade-offs

Approach	How it uses the vitals	What to check
Constant probability	Training share of septic patients	First log-loss reference
Logistic regression	Explicit summaries of vitals	Scaling, linear score, regularization
Tree ensembles	Interactions among summaries	Overfitting, probability quality
RNN / GRU	Learned memory of hourly vitals	Window length, tuning, variability
Transformer	Attention across hours	Position codes, data, compute
Fair comparison	Same patients, input window, labels and metric.	

Validation: development splits and a final test



Choose the question	New patients: group by patient. Future admissions: split by time.
Combine if needed	New patients in the future: enforce both group and time separation.
Develop freely	Choose / revise train–validation splits; inspect statistics and errors.
Freeze, then test	Refit on train + validation; apply the frozen pipeline to test.

Data leakage: three boundaries to protect

Prediction cutoff	Inputs must be available when the prediction is made. Example: a diagnosis first recorded after hour 23.
Evaluation unit	Keep related records together when evaluating unseen people. Hourly rows and repeat stays are the same grouping issue.
Final test	Keep test statistics, labels and feedback out of development. Example: fitting a scaler on train + test.
Validation is development	Using validation to improve a model is expected; its score can become optimistic.

Evaluation: log loss

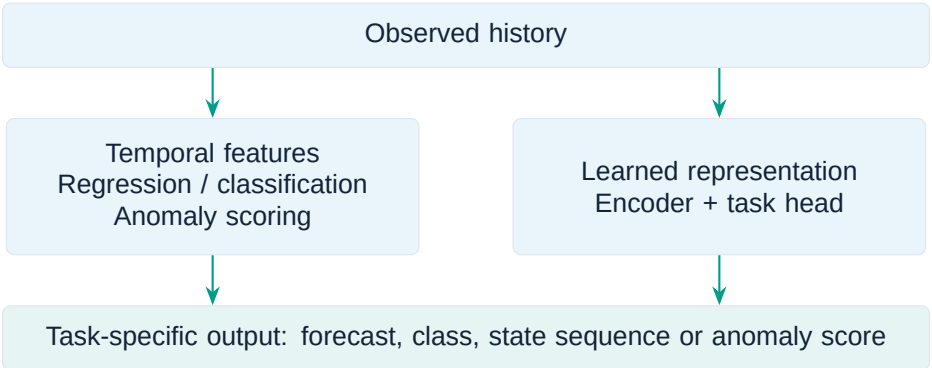
$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log p_i + (1-y_i)\log(1-p_i)\right].$$

Sepsis? (y)	Predicted $p(y=1)$	Decision at 0.5	Loss
1 (sepsis)	0.60	1	0.511
1 (sepsis)	0.90	1	0.105
0 (no sepsis)	0.90	1	2.303

Evaluation: ranking, calibration and errors

Log loss	Probability quality; lower is better.
Ranking	ROC-AUC; PR-AUC for rare positive events.
Calibration	Predicted 20% risk $\longrightarrow$ about 20% observed events?
Decision errors	Precision / recall at a chosen threshold.
Slices	Performance by age group, ward, or admission period.

Summary: two ways to represent a history



Group project: predict a two-week pause



One example	One student in one module presentation.
Output	$P(\text{zero clicks on days 29–42})$.
Example tables	`train.csv`: labeled. `test.csv`: unlabeled.
History tables	`vle_events.parquet`: clicks by day/type. `past_assessments.parquet`: past submissions.

Group project: file formats and loading

Format	Strengths	Costs / risks	pandas
CSV	Text, portable, diff-friendly	Slow; types inferred	`read_csv`
Parquet	Typed, compressed, columnar	Binary; needs `pyarrow`	`read_parquet`
JSON	Small nested records	Whole file in memory	`read_json`
JSONL	One record per line	No schema; per-line parse	`read_json (lines=True)`
Pickle	Fast local DataFrame cache	Unsafe; version-fragile	`read_pickle`

```
train = pd.read_csv("train.csv")
vle   = pd.read_parquet("vle_events.parquet",
                        columns=["sample_id", "day", "clicks"])
asm   = pd.read_parquet("past_assessments.parquet")
sub   = pd.read_csv("sample_submission.csv")
```

Group project: starting questions

Representation	Which parts of the history might matter?
Baseline	What is your simplest probability prediction?
Validation	How would you handle later presentations and repeated students?
Improvement	Does the change improve validation log loss?
Instructions	**Moodle**: project brief + invitation. **Kaggle**: data + submission instructions.

Further reading

Course project	*Predicting a Short-Term Learning-Platform Pause*: the student instruction PDF and announcements on HKU Moodle.
Features and splits	scikit-learn: cross-validation and grouped splits; model evaluation.
Data and formats	pandas: IO tools for CSV, Parquet, JSON, JSONL and Pickle.
Sequence models	PyTorch: GRU; TransformerEncoder; Vaswani et al. (2017), Attention Is All You Need.
Forecasting background	Forecasting: Principles and Practice, ARIMA, for the forecasting contrast on the method slides.

Contact: Yinghao Zhu (yhzhu99@connect.hku.hk)